

The Tallyman: an overnight, fully agentic build of an 8-bit text adventure

Oct 6, 2026 · @Nenad Mirkov

SPOILERS: play the game first

This paper gives away how the mystery ends: the killer, the finale and the puzzles. Play THE TALLYMAN before reading on.

Summary

From a prompt typed in about four minutes, a team of AI agents built and shipped a complete C64/ZX Spectrum-style crime thriller text adventure overnight, with no human decision along the way. Active build time was about 4.5 hours, from 23:33 to 03:42 and then 08:30 to 08:52, with one forced pause for usage limits.

Measure	Result
Tickets	38 (25 planned, 13 bug and fix tickets)
Commits / tags	56 commits; tags plan-approved, m0, m1, m2, m3, v1.0
Agent runs	43 sub-agent runs in 6 roles, plus 7 Codex review runs
Code	16,300 lines of source (engine, UI, audio, content, art), 14,500 lines of tests
Design docs	4,300 lines: plan, engine contract, story bible
Automated tests	1,886, plus a browser smoke test (8/8)
Defects caught before release	7 plan blockers, 13 code blockers, 4 softlocks, 1 high and 23 smaller playtest issues
Blind playtest	3 AI testers with no access to the code; all 3 won; rated 8/10
Release	one offline HTML file, about 680 KB, tagged v1.0 at 08:52

The orchestrator (Claude, the main session) wrote the plan and the tickets, dispatched agents, verified every result independently and committed. It never trusted an agent's own report. Codex reviewed the plan and the code until it ran out of credits mid-build; an independent Claude reviewer covered the gap, and Codex returned for the final release review.

The prompt

The whole project started from one message, typed in about four minutes and then left alone. Verbatim, typos included:

this is a hobby project: I want you to totally autonomosly build me an text adventure game. Since I'm 1975 I grew up on commodore 64 and zx spectru. I always wanted to build a text adventure game. It should be a crime thriller with a little bit of horror theme. Some 8bit sound would be great, and textual graphics... But the idea here is that you do this fully autonomosly. And I want a fully agentic engineering approach. First I want you (harness) to create a very good plan, then run codex until he approves the plan. Then build agents in this local projects. Organize the work and watch for context window. Create statuses and handover whenever context goes above 50% | create docs file, and create me an html with the plan and progress as things get built. create local repo, and commit everytime you reach some milestone. create project with tickets (local folder), and organize work with agents where ticket by ticket gets executed. no human in the loop at all. can you do it? I only want this one interaction, and then you are on your own

Each request in it turned into a concrete mechanism:

Asked for	How it was delivered
A very good plan, approved by Codex	<code>docs/PLAN.md</code> ; 3 Codex review rounds (5, then 2 blocking issues) until APPROVED WITH NOTES
Build agents in this project	6 role definitions in <code>.claude/agents/</code> : engine-dev, ui-dev, audio-dev, writer, artist, qa
Tickets in a local folder, executed one by one	<code>tickets/TT-xxx-slug/</code> with front-matter status; one agent per ticket, 38 tickets
Watch the context window; handover above 50%	<code>docs/HANDOVER.md</code> rewritten at every milestone and before risky points (6 times)

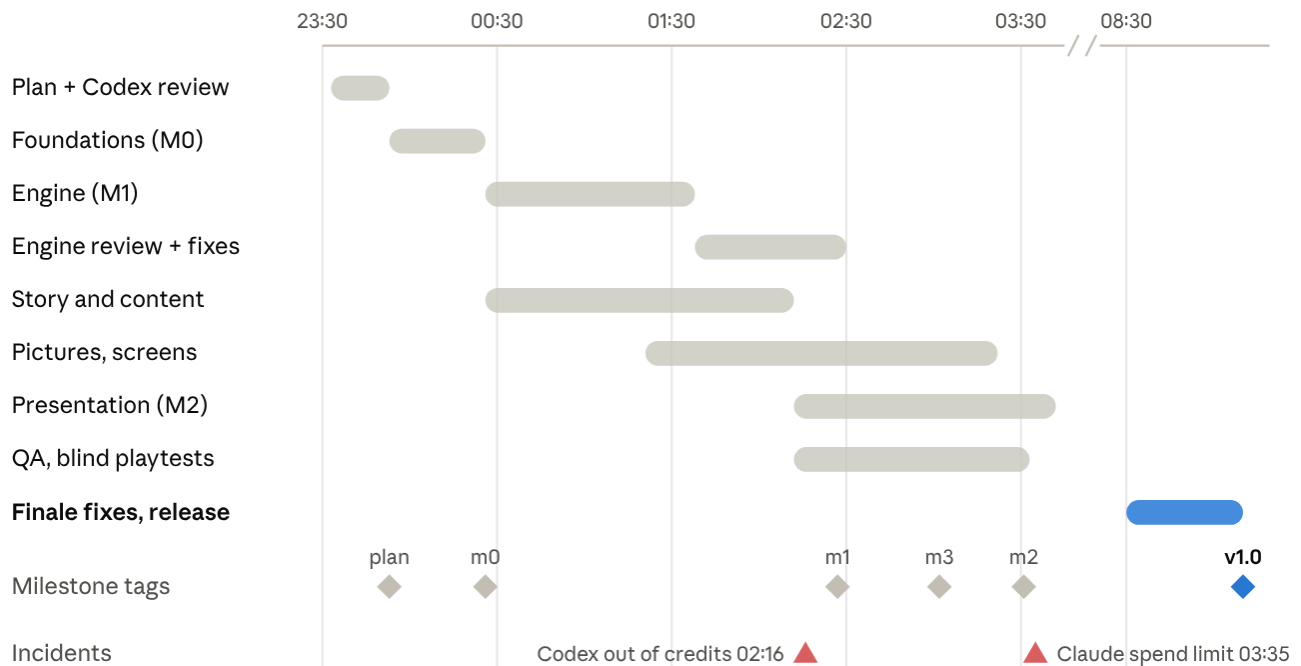
Asked for	How it was delivered
Statuses and docs	<code>docs/STATUS.md</code> , <code>DECISIONS.md</code> (13 decisions), <code>REVIEWS.md</code>
An HTML page with plan and progress	<code>tools/progress.js</code> regenerates <code>docs/progress.html</code> before every commit
Local repo, commit at milestones	56 commits, each ticket verified in a clean git worktree first; 6 tags
Crime thriller with some horror, 8-bit sound, textual graphics	Blackmere 1984 story, SID-style WebAudio synth, 42 PETSCII location pictures
No human in the loop	No questions were asked. Hard limits were kept: no pushes, no paid actions, no destructive git

Timeline of the night

The plan was approved 20 minutes after the prompt, the engine was complete at 01:38, and the game was winnable end to end by 02:12. Most of the rest of the night went into polish, reviews and playtests.

About 4.5 hours of active build, from prompt to v1.0

Night of 5 to 6 October 2026; the time axis is cut from 03:45 to 08:30 (usage pause, laptop asleep)



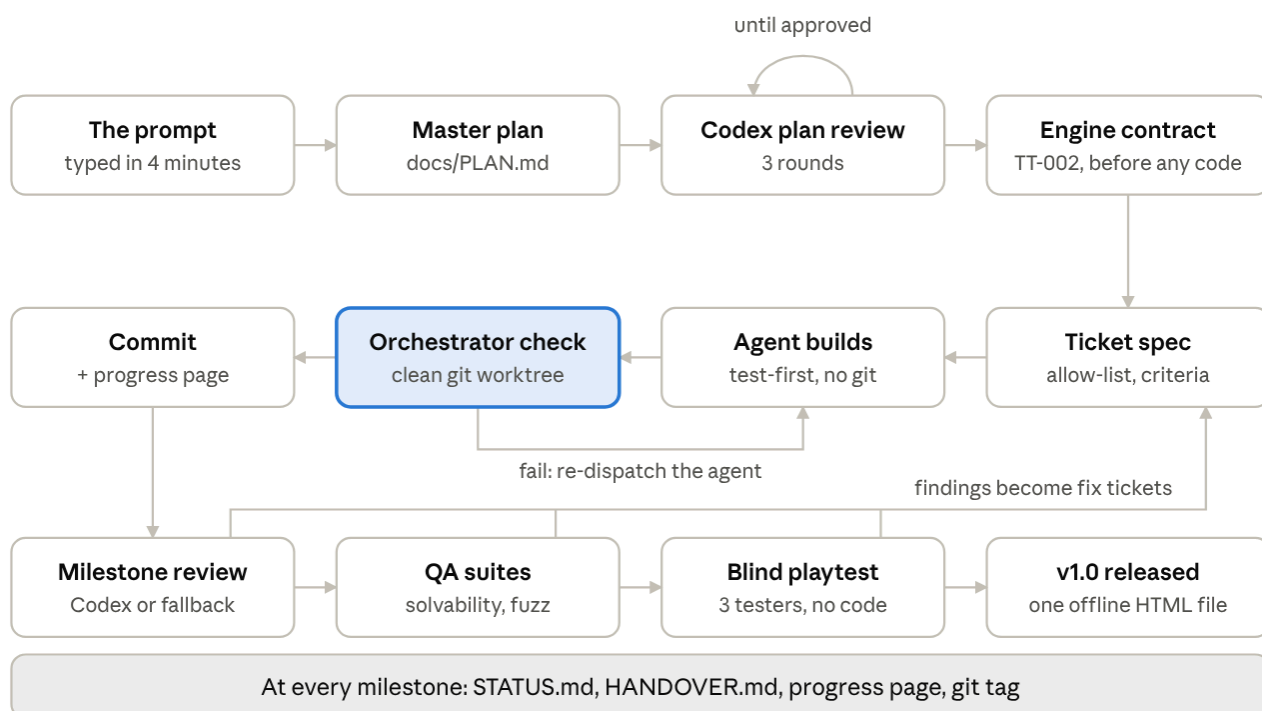
git commit and tag times from the repo · phases are approximate spans from first dispatch to last commit

Phases overlap because up to 6 agents ran in parallel on disjoint files. The milestone tags are not in numeric order: content (m3) was approved before presentation (m2), because the presentation review needed a second round.

How the work was organised

One orchestrator (the main Claude session) owned the plan, the tickets, git and every quality gate. Role agents did the building, each on one ticket at a time, with up to 6 running in parallel.

Every ticket passed an independent check before it was committed



the delivery process as run on the night · 12 steps, 3 feedback loops

The check in the middle row is the heart of it. An agent's report was never trusted: the orchestrator re-ran the full check in a fresh git worktree at the new commit, looked at screenshots for UI work and played the game itself where it mattered.

Role	Tickets	Model	What it owned
engine-dev	13	Opus (Sonnet for 3)	parser, world model, actions, daemons, engine fixes
writer	10	Opus	story bible, all prose and content data, playtest fixes

Role	Tickets	Model	What it owned
ui-dev	7	Opus (Sonnet for 1)	canvas screen, terminal, boot, integration, smoke tests
artist	4	Opus	42 location pictures, title and 6 ending screens
qa	2	Opus (testers: Opus, Sonnet)	solvability, save-determinism and fuzz suites; blind playtests
audio-dev	1	Opus	SID-style synth, music, effects, audio audit
orchestrator	1	Opus	plan, tickets, verification, commits, docs, this paper
Codex (OpenAI)	n/a	gpt-6-astra, high	plan review and code review, read-only
Fallback reviewer	n/a	Claude Fable	4 reviews while Codex had no credits

The SDLC in numbers

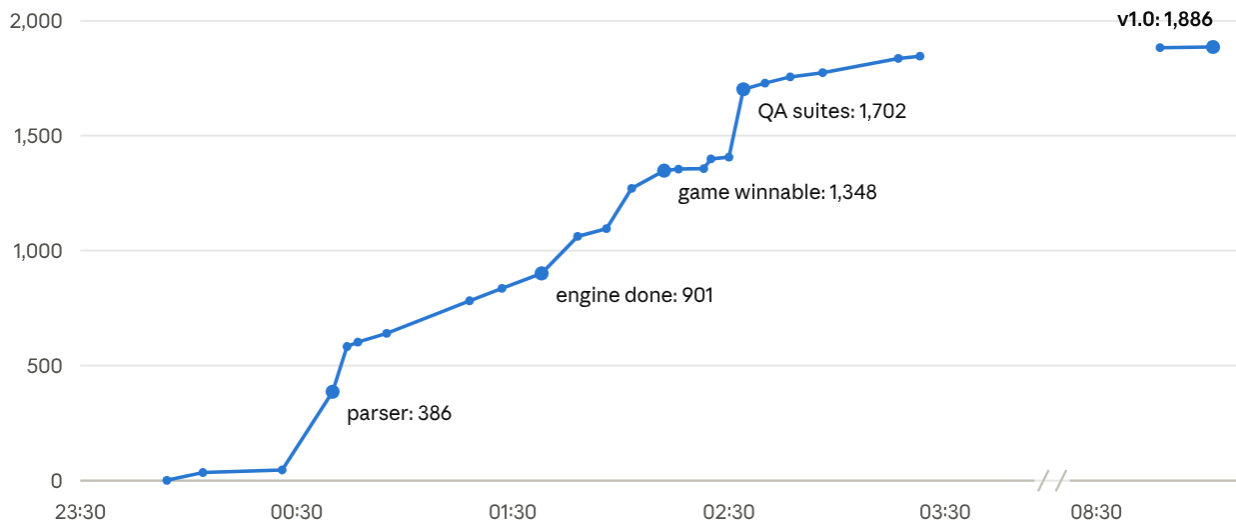
The plan's 25 tickets in 5 milestones held up; 13 more tickets were added for review findings, softlocks and playtest fixes.

Milestone	Tickets	Delivered	Review outcome
Plan	0	PLAN.md , agent roles, process	Codex: 2 rounds of changes, approved in round 3
M0 Foundations	4	scaffold, engine contract, canvas screen, progress page	Approved by the orchestrator (contract gate)
M1 Engine	6 + 3 fix	parser, resolver, world, actions, NPCs and case, daemons	Codex: 9 then 2 blocking defects; approved in round 3
M2 Presentation	4 + 2 fix	terminal, integration, audio, boot sequence	Fallback reviewer: 2 blocking defects; approved in round 2
M3 Content	8 + 6 fix	story bible, 42 rooms, art, QA suites	Fallback reviewer: approved with 13 notes; QA found 4 softlocks

Milestone	Tickets	Delivered	Review outcome
M4 Release	3 + 2 fix	smoke tests, blind playtest, README	Codex final review: approved, 0 blocking

Automated tests grew from 1 to 1,886 overnight

Passing tests at each verified commit; time axis cut from 03:45 to 08:30



test counts from the clean-worktree check at each commit (orchestrator log)

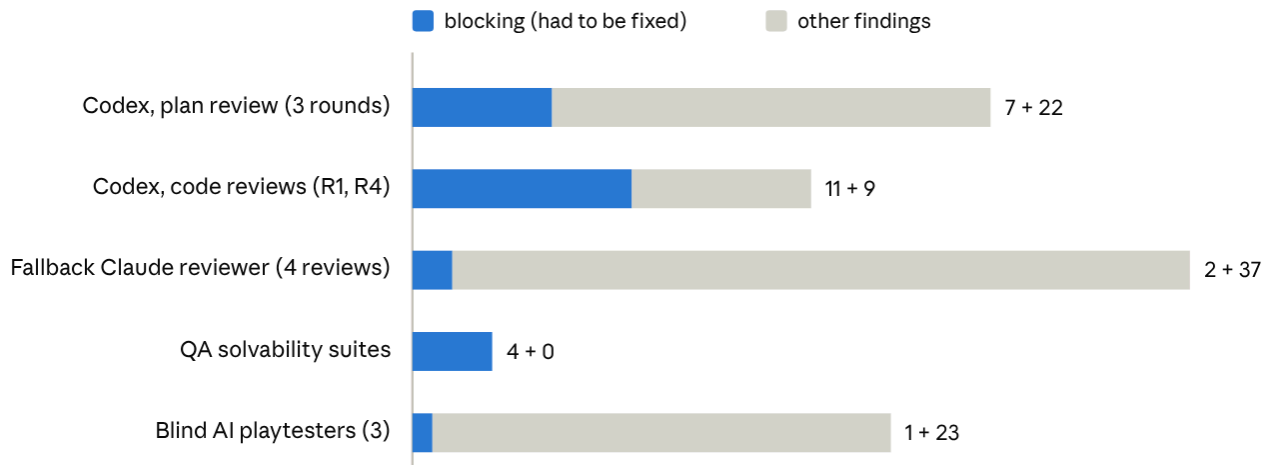
Tests were written alongside each ticket, not at the end. The two biggest jumps are the parser's 340 test cases and the QA suites' 295 solvability, determinism and fuzz tests.

Mistakes and how they were caught

25 problems serious enough to block a release were caught before v1.0, and no single check would have found them all. Each kind of check found a different kind of mistake.

25 release blockers came from five different kinds of check

25 blocking findings and 91 other findings (notes, friction) before v1.0



counted from reviews/*.md, docs/QA-REPORT.md and the TT-130/131 implementation notes

Caught by	Typical mistake it found	Example
Codex plan review	gaps in the design	The finale's rules were ambiguous enough to allow different games; resources could run out with no recovery
Codex code review	engine defects with exact reproductions	A critical item could be destroyed by throwing the bag it was in; a nested death could be overwritten by a later ending
Fallback reviewer	presentation bugs a player would hit	The typewriter effect was skipped after QUIT and a new game; the title needed two key presses
QA solvability suites	unwinnable states	A torch dropped in a dark room could never be found again; leaving the finale room without the torch trapped you
Blind playtesters	unfair puzzle design	Nothing pointed to HANDCUFF PIKE in the finale; both testers died there

All 25 were fixed before release. The 7 plan blockers were fixed in the plan; every code fix came with a regression test that replays the failing case. The orchestrator also caught smaller things itself while verifying, such as `X ME` not being understood and a build that could not bundle the art.

The most telling result: about 1,800 automated tests were green while the finale was still unfair to a first-time player. Only a blind playtest, by agents who had never seen the code, could see it.

Incidents during the night

Six things went wrong outside the code itself. None stopped the build for long, but each one taught something.

Time	Incident	Impact	What happened next
03:45 to 08:30	Laptop went to sleep with an agent mid-ticket	Agent suspended for hours; its edits sat uncommitted	It resumed on wake and finished; the orchestrator checked the working tree before committing anything
03:35	Claude spend limit hit ("individual spend limit", reset 03:40)	2 running agents (TT-130/131, TT-132) killed before writing anything	Committed what was verified, wrote a handover, resumed after the reset and re-dispatched both
02:25	A UI agent stopped its dev server with <code>pkill -f tools/serve.js</code>	Could have killed other agents' servers (none were running)	Rule added to the ui-dev role: stop processes by PID only
02:16	Codex workspace out of credits during engine review round 3	No cross-vendor review for M1 round 3, M2 and M3	Decision D-009: an independent Claude reviewer on a different model (Fable, fresh context, read-only). Codex probed at each milestone; back for the final review
02:15	HEAD briefly red: a strict-lint test expected art that was still in flight from a parallel agent	One commit failed the clean-worktree check	Test relaxed until all art landed, then made strict again; HEAD green within 2 minutes

Time	Incident	Impact	What happened next
02:00	The orchestrator's own wait loop used <code>pgrep -f</code> on a pattern that appeared in its own command line	Loop never ended; noticed only because the review file already existed	Killed it; from then on, wait on the exact process ID

Lessons learned

The short prompt worked because it described a process (plan, external review, tickets, agents, handover, commits), not a product. The process carried the quality; the product details came from the plan.

What worked and should be repeated

- Contract before code.** TT-002 (`ARCHITECTURE.md` , 1,372 lines) fixed state, events, turn order and the content language before any consumer was built. That made it safe to run up to 6 agents in parallel with no merge conflicts.
- File allow-lists per ticket, shared files owned by the orchestrator.** Agents never touched each other's files and never ran git. The orchestrator staged only the ticket's files.
- Verify in a clean worktree, not in the working tree.** Every commit was re-checked in a fresh `git worktree` at HEAD. This caught the one red HEAD that a local check would have hidden behind uncommitted work.
- Different lenses find different bugs.**
 - Codex found engine defects with exact reproductions (11).
 - The solvability suites found 4 softlocks that no reviewer saw.
 - Blind playtesters found the unfair finale, which 1,800 passing tests could not see.
 - For a game, a blind playtest is not optional.
- Small, well-specified tickets.** When the spend limit killed two agents, almost nothing was lost: work was committed per ticket, and the specs were already on disk.

When credits run out

Both limits hit the same night: Codex at 02:16, Claude at 03:35. The playbook that emerged:

- Probe before you depend.** A one-line `codex exec` probe at each milestone tells you within seconds whether the reviewer is there.

2. **Fall back, don't stall, and log it.** The fallback was an independent reviewer on a different model, in a fresh context, read-only, with the same prompt and verdict format. It is recorded as a decision (D-009) so nobody mistakes it for cross-vendor review.
3. **Owe a catch-up.** When the primary reviewer returns, it reviews everything it missed. Codex's final review covered all of that and found 0 blockers.
4. **For the builder's own limit:** commit verified work immediately, keep the handover current, and re-dispatch killed agents from their tickets.
 - Before an overnight run, check the spend limit and keep the laptop awake (`systemd-inhibit` or `caffeinate`), or run on a server.
 - Raising a limit is a cost decision, so the agent should never make it on its own.

Was the "handover at 50% context" idea useful?

Yes, but not for the reason it was given.

- **Context never got close to the limit.** Agents did the heavy reading and returned summaries of 200 words or fewer, so the orchestrator stayed lean. Delegation managed the context, not the handover.
- **A percentage threshold is hard to apply.** An agent cannot measure its own context precisely.
- **The handover still proved essential, for interruptions.** After the spend-limit pause and the laptop sleep, a one-word "continue" was enough, because `HANDOVER.md` named the exact commit, the agents in flight and the next steps.

Recommendation: keep the handover, but trigger it on events rather than a percentage: every milestone, before anything risky, and before any pause. Treat it as a fixed checklist (HEAD, dirty files, agents in flight, last failure, next steps).

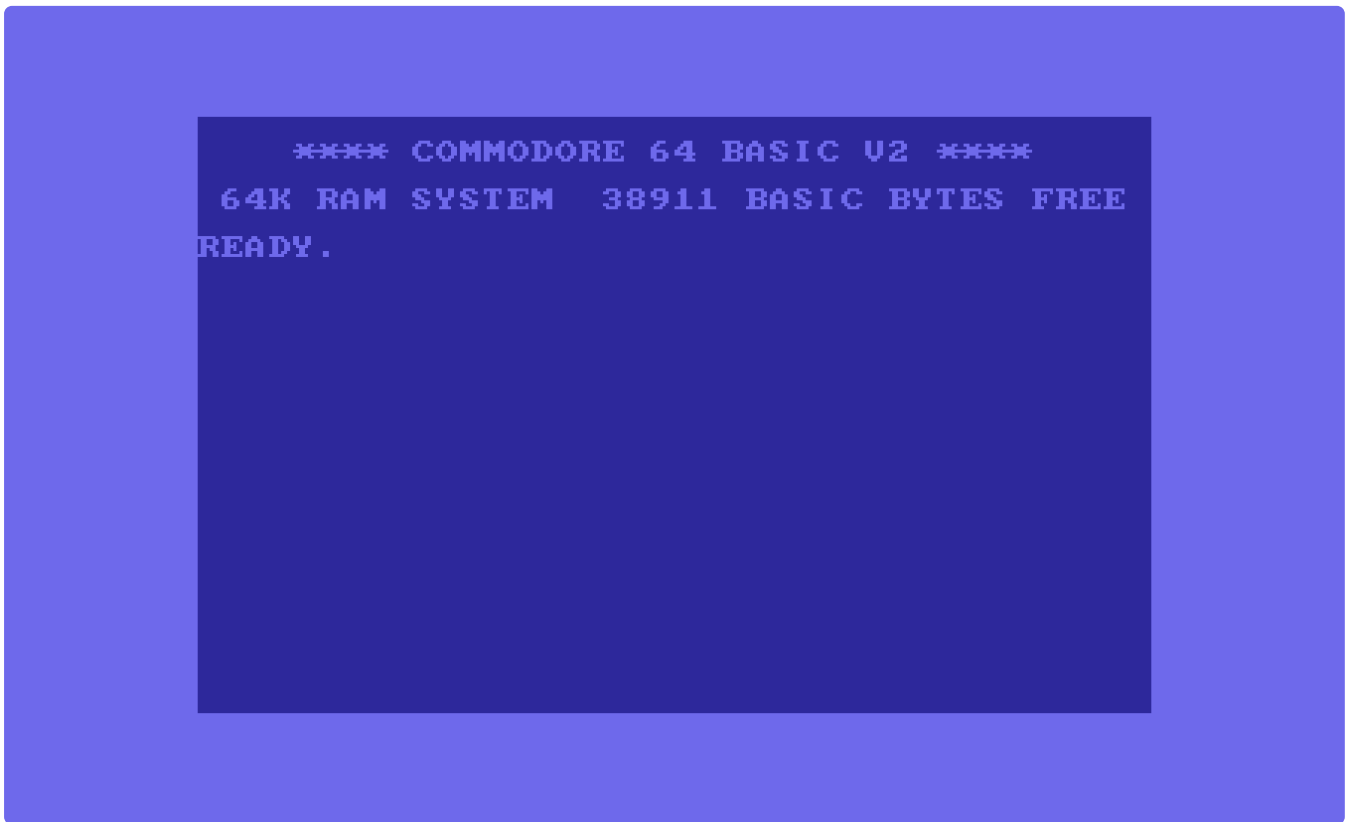
What to change next time

- Have a human audition the audio early. Agents can only measure level, pitch and length, not whether it sounds good.
- Run a short blind playtest of the finale before content is called done, not after.
- Add touch and phone-keyboard automation to the smoke suite (gap D-013).

Screenshots

All screenshots come from the release build running in headless Chrome. Agents captured them as evidence during verification.

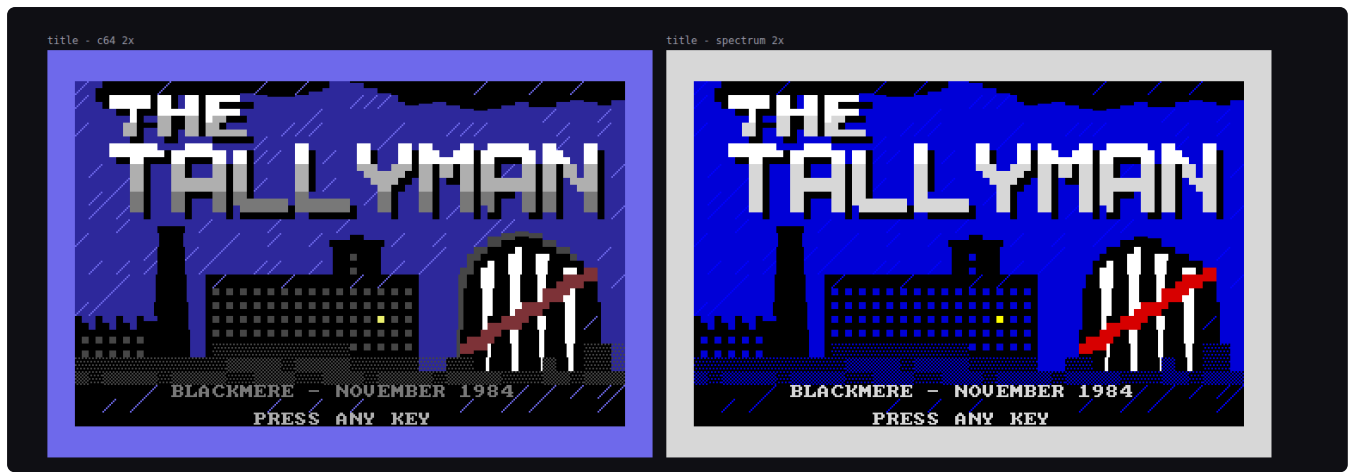
1. Power-on. The game boots like a real Commodore 64. Pressing a key to "press PLAY on tape" also unlocks the browser's audio.



2. Tape loading. Border stripes and tape screech while the title loads in row by row.



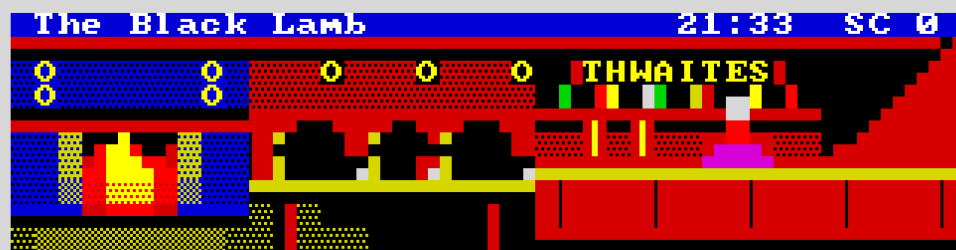
3. Title screen. A block-letter logo, the mill, and the standing stone with four strokes and the red fifth. The C64 and Spectrum themes are shown side by side.



4. In game, C64 theme. The status bar shows location, clock and score, with a PETSCII picture above and the text below.



5. In game, ZX Spectrum theme (`THEME SPECTRUM`).



Exits: north, east, south, west, in.

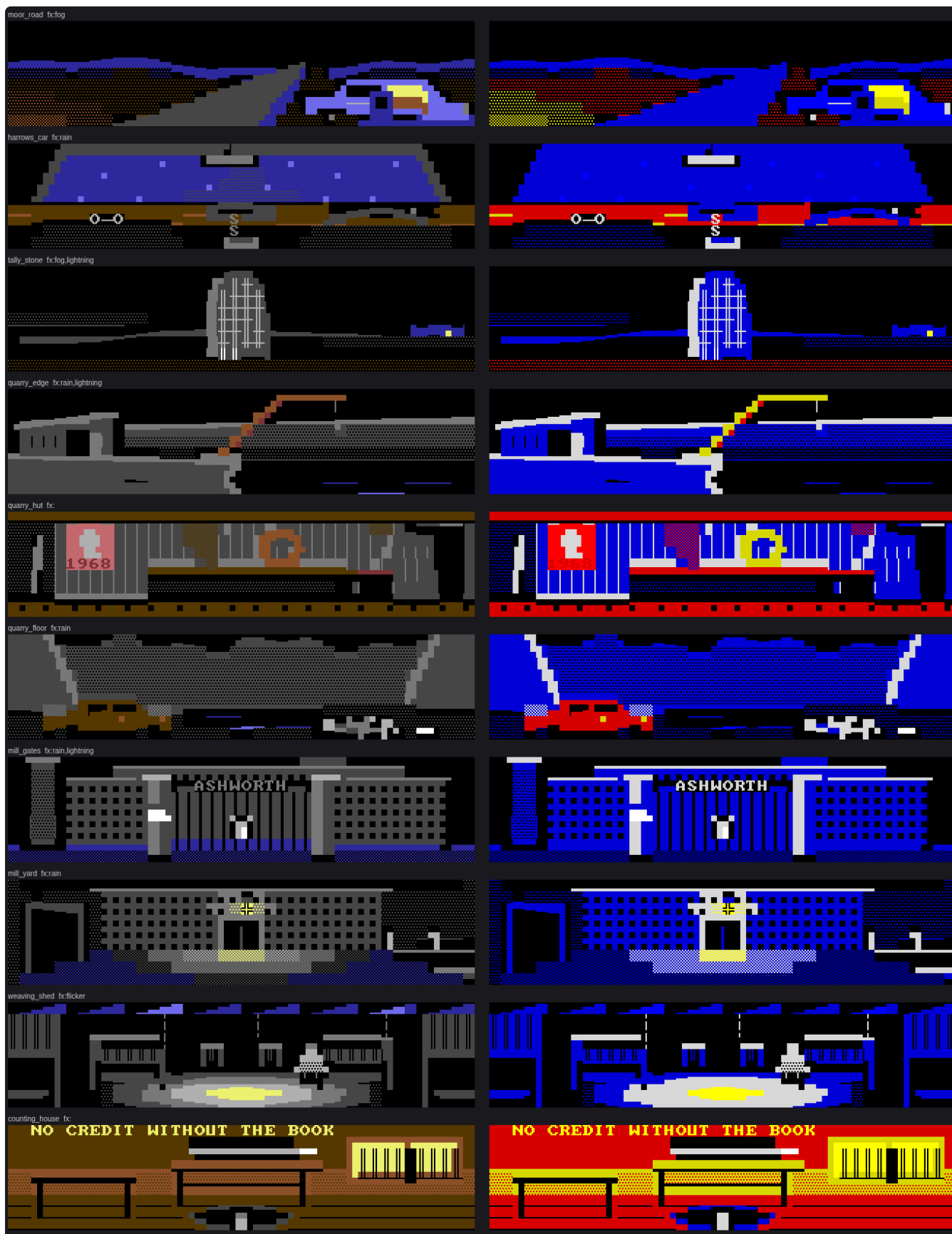
> w

The Black Lamb

The Black Lamb: low beams, horse
brasses, a fire that has seen better
winters. Three regulars stare into their
mild and do not look up. Behind the bar
a stair climbs to the guest rooms.
Market Square is east.
Maggie is behind the bar, polishing a
glass that is already clean.
Exits: east, up.

>

6. Location art. Part of the 42-picture set, in C64 and Spectrum palettes.



7. An ending screen. This is what happens if you accuse the wrong suspect.



8. The progress page you asked for, generated from the tickets and git history, at release.

**** COMMODORE 64 BASIC V2 ****

THE TALLYMAN

BUILD PROGRESS · AUTONOMOUS AGENT BUILD

UPDATED 2026-10-06 06:52 UTC PHASE V1.0 RELEASED (TAG `V1.0`), DONE 38/38 (100%)

READY.

Milestones

Board

Status

Plan

Reviews

Commits

Screenshots

Milestones

M0

Foundations

4/4 done 4 done

M1

Engine

6/6 done 6 done

M2

Presentation

4/4 done 4 done

M3

Content

8/8 done 8 done

M4

Polish & release

3/3 done 3 done

BUG

Bug fixes

13/13 done 13 done

Ticket board

TODO

0

none

IN-PROGRESS

0

none

VERIFY

0

none

DONE

38

TT-001

Scaffold project with working check pipeline and minimal single-file build

engine-dev sonnet

TT-002

Define the engine contract (ARCHITECTURE.md + types.js)

engine-dev opus needs TT-001

TT-003

Build the 40x25 canvas character screen with font8x8 and C64/Spectrum palettes

ui-dev opus needs TT-001

BLOCKED

0

none

Appendix: where everything lives

All paths are relative to the repo `~/workspace/sandbox/old-adventure-game`, which is local only and was never pushed.

Artefact	Path
The game (one offline file)	<code>dist/tallyman.html</code> (after <code>npm run build</code>)
How to play, how it was built	<code>README.md</code>
Approved master plan	<code>docs/PLAN.md</code>
Engine contract (40 decisions, C1 to C40)	<code>docs/ARCHITECTURE.md</code>
Story bible, walkthrough, softlock audit (spoilers)	<code>docs/STORY.md</code>
Orchestrator decisions D-001 to D-013	<code>docs/DECISIONS.md</code>
Every review, verdict and resolution	<code>docs/REVIEWS.md</code> (raw transcripts in <code>reviews/</code> , git-ignored)
QA, smoke and playtest reports	<code>docs/QA-REPORT.md</code> , <code>docs/SMOKE-REPORT.md</code> , <code>docs/PLAYTEST-REPORT.md</code> , <code>docs/playtest/</code>
Live status and resume point	<code>docs/STATUS.md</code> , <code>docs/HANDOVER.md</code>
Progress dashboard	<code>docs/progress.html</code> (<code>npm run progress</code>)
Tickets with spec and implementation notes	<code>tickets/TT-xxx-*/</code> and <code>tickets/BOARD.md</code>
Agent role definitions	<code>.claude/agents/*.md</code>
Art gallery, audio audition	<code>docs/art-preview.html</code> , <code>tools/audio-demo.html</code>